

课程笔记

从渗透测试到通用问题的求解路径：Cairn AI 系统设计与实践

Bytex 战队 / 腾讯安全众测

2026 年 5 月 7 日



视频作者/频道：腾讯安全众测

发布日期：2026-05-01

视频时长：16 分 56 秒

视频链接：<https://www.bilibili.com/video/BV1yQ9YBNEuZ/>

目录

1 问题本质：渗透测试作为无限状态空间中的有向搜索	3
1.1 从信息收集到状态空间搜索的范式转换	3
1.2 无限状态空间与有向搜索的形式化定义	4
1.3 通用问题求解引擎的设计起点	6
1.4 本章小结	7
2 黑板架构：共享知识空间的设计范式	7
2.1 侦探围坐黑板的隐喻与黑板架构的历史	7
2.2 Cairn 黑板的三类核心元素：Factor、Intent、Hint	9
2.2.1 Factor（关键事实）：已确认的板书	9
2.2.2 Intent（探索意图）：粉笔画出的问号	9
2.2.3 Hint（外部提示）：便利贴上的线索	10
2.3 从黑板到图：知识结构的图化表示	10
2.3.1 知识图的形式化定义	10
2.3.2 从起点到终点的有向路径	11
2.3.3 Cairn（路标）命名的意境	11
2.4 本章小结	12
3 Agent 工作循环：OODA 与三任务类型	13
3.1 三类任务类型：Bootstrap、Reason、Explore	13
3.1.1 Bootstrap：直接尝试解决	13
3.1.2 Reason：读取全图并推理下一步	13
3.1.3 Explore：执行推理出的路径	14
3.2 OODA 循环：观察、分析、决定、执行	14
3.2.1 三类任务与 OODA 的映射	14
3.2.2 循环的数学表达	15
3.3 真实题目解题过程演示	15
3.3.1 起点与终点的定义	15
3.3.2 阶段一：Bootstrap 尝试	15
3.3.3 阶段二：Reason-Explore 循环	16
3.3.4 复杂题目的真实图结构	16
3.4 本章小结	17
4 系统工程架构：Current Server、Dispatcher 与容器化 Agent	18
4.1 三层工程架构：Current Server、Dispatcher、容器	18
4.1.1 Current Server：协议层与黑板层	18
4.1.2 Dispatcher：读图与写图的中间层	19
4.1.3 容器：搭载各类 Agent 的执行环境	19
4.2 Hint 注入机制与人类指挥官角色	20
4.2.1 Hint：图外的高维输入	20
4.2.2 任务式指挥：Mission Command	20
4.3 依赖处理与并发执行策略	21

4.3.1	任务依赖：由 AI 推理动态产生	21
4.3.2	串行推理与并行执行	21
4.3.3	串并混合的执行模型	22
4.4	本章小结	23
5	设计哲学：Less is More 与涌现式智能	23
5.1	传统多 Agent 架构 vs Cairn 架构的核心差异	23
5.1.1	传统架构：设计时预定义的角色分工	24
5.1.2	Cairn 架构：运行时动态产生的任务	25
5.1.3	分工来源的本质差异	25
5.2	蚁群隐喻：信息素间接协调与涌现行为	26
5.2.1	蚂蚁的间接协调机制	26
5.2.2	Cairn 与蚁群的同构性	26
5.2.3	信息素浓度的数学表达	27
5.3	Less is More：去除而非堆叠的设计哲学	27
5.3.1	一个令人震惊的事实	27
5.3.2	寻找本质与最小表达	28
5.3.3	去掉比加上更难	29
5.4	赛场表现、评委问答与技术反思	29
5.4.1	赛场表现：AK 全场的背后	29
5.4.2	评委问答一：外层严格协议 vs 提示词约束	30
5.4.3	评委问答二：模型选择	31
5.4.4	评委问答三：依赖关系与并行执行	31
5.5	本章小结	31
6	总结与延伸	32
6.1	演讲者的实质性总结	32
6.2	结构化提炼	33
6.2.1	核心论点	33
6.2.2	核心机制	33
6.2.3	实践启示	33
6.3	概念压缩与跨章节关联	34
6.3.1	核心概念的层次结构	34
6.3.2	跨章节关联	34
6.4	谨慎的泛化	34
6.5	具体的收获与开放问题	35
6.5.1	具体收获	35
6.5.2	开放问题	35
6.5.3	下一步	36

1 问题本质：渗透测试作为无限状态空间中的有向搜索

本章定位

本章回答一个根本问题：渗透测试在数学上究竟是什么？我们将从安全圈耳熟能详的“信息收集”出发，把它提升到一个更普适的形式化视角——无限状态空间中的有向搜索。这个视角不仅是后续系统设计的理论基石，也是理解 Cairn（衍迹）系统为什么能同时适用于渗透测试、漏洞挖掘乃至数学证明的钥匙。

1.1 从信息收集到状态空间搜索的范式转换

直觉先行：一个经典回答的再审视

安全圈有一个被广泛认同的经典判断：

渗透测试的本质是信息收集。

这句话没错，但它只回答了一半。它告诉我们每一步在做什么——扫描端口、收集目录、分析响应——却没有回答整个过程在做什么。如果把每一次信息收集看作一次独立的动作，我们就容易陷入“工具链思维”：买更好的扫描器、写更全的字典、堆更多的 POC。这种视角下，渗透测试变成了一场对工具的军备竞赛。

但如果我们换一个角度追问：

核心追问

信息收集的结果去了哪里？它如何影响下一步的决策？

答案呼之欲出：每一次信息收集，都是从当前已知出发的一次探索；探索的结果，又变成新的已知。已知在累积，选择空间在变化，路径在动态展开。这不是静态的工具调用，而是一个状态在演化、空间在展开的动态过程。

从“动作”到“状态转移”

把上述直觉形式化，我们得到以下状态转移模型：

$$s_{t+1} = \delta(s_t, a_t) \quad (1)$$

其中各符号含义如下：

- s_t ：时刻 t 的系统状态，包含截至当前已收集的全部信息（开放端口、目录结构、凭证、已利用的漏洞等）。
- a_t ：时刻 t 采取的行动，即一次信息收集动作（如端口扫描、SQL 注入尝试、社会工程）。
- δ ：状态转移函数，描述行动如何将当前状态映射为下一状态。
- s_{t+1} ：执行行动后的新状态，包含原有信息加上本次探索的新发现。

常见误解

不要把 s_t 简单理解为“一个 IP 地址”或“一台主机”。状态 s_t 是认知状态——渗透测试者对目标系统当前所掌握的全部知识的集合。两台主机、相同的端口开放情况，如果渗透者掌握的其他信息不同（如已知凭证、已知漏洞），则属于不同的状态。

这个形式化的关键洞察在于：**信息收集不再是目的本身，而是状态空间中的移动方式**。每一次扫描、每一次尝试，都是在状态空间中迈出的一步。渗透测试的整体结构，因此可以被重新诠释为：

范式转换

渗透测试 = 在状态空间中寻找从起点到终点的可行路径。

1.2 无限状态空间与有向搜索的形式化定义

直觉先行：为什么“无限”且“有向”

想象你站在一片迷雾森林的入口。你面前有无数条小径，每条小径又分叉出更多小径，你永远无法事先画出完整的地图——这就是无限。但你心里清楚，森林的某处有一座灯塔，你的唯一目标就是抵达那座灯塔——这就是有向。渗透测试正是如此：你面对的是不可完全枚举的可能性，但你的终点从一开始就是明确的。

形式化定义

将上述直觉严格化，渗透测试可以形式化为一个四元组：

$$\mathcal{P} = (\mathcal{S}, s_0, s^*, \delta) \quad (2)$$

其中各符号含义如下：

- $\mathcal{S}$ ：状态空间（state space），包含渗透过程中可能遇到的所有认知状态。 $|\mathcal{S}| \rightarrow \infty$ ，即状态空间是无限的——你无法在开始前穷举所有可能遇到的情况。
- $s_0 \in \mathcal{S}$ ：初始状态（initial state），即渗透测试的起点，通常包含目标 IP、域名、题目描述等已知信息。
- $s^* \in \mathcal{S}$ ：目标状态（goal state），即渗透测试的终点，如获取 Shell、拿到 Flag、完成权限提升。 s^* 在开始时即被明确定义。
- $\delta: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ ：状态转移函数，如前所述，描述行动如何驱动状态演化。

在此基础上，一次成功的渗透测试对应于状态空间中的一条解路径：

$$\pi = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s^*) \quad (3)$$

即一个从 s_0 出发、经由一系列行动 $\{a_t\}$ 、最终抵达 s^* 的有限序列。

核心概念：无限与有向

无限 ($|\mathcal{S}| \rightarrow \infty$)：渗透过程中遇到的情况不可完全枚举，每个发现都可以展开全新的维度。你不可能事先准备一个覆盖所有场景的剧本。

有向 (s^* 预先确定)：虽然中间路径未知，但终点从一开始就是明确的。搜索不是漫无目的的游荡，而是朝着既定目标的定向探索。

信息收集的角色再定位

在这个形式化框架下，信息收集的角色被精确地重新定位：

信息收集 $\equiv$ 状态空间中的移动方式 (4)

每一次信息收集动作 a_t 都是一次状态转移的尝试。它可能成功（发现新信息，进入新状态），也可能失败（没有新发现，状态基本不变），还可能触发副作用（如触发 WAF、被封禁 IP，进入意料之外的状态）。无论结果如何，它都在状态空间中留下了一条轨迹。

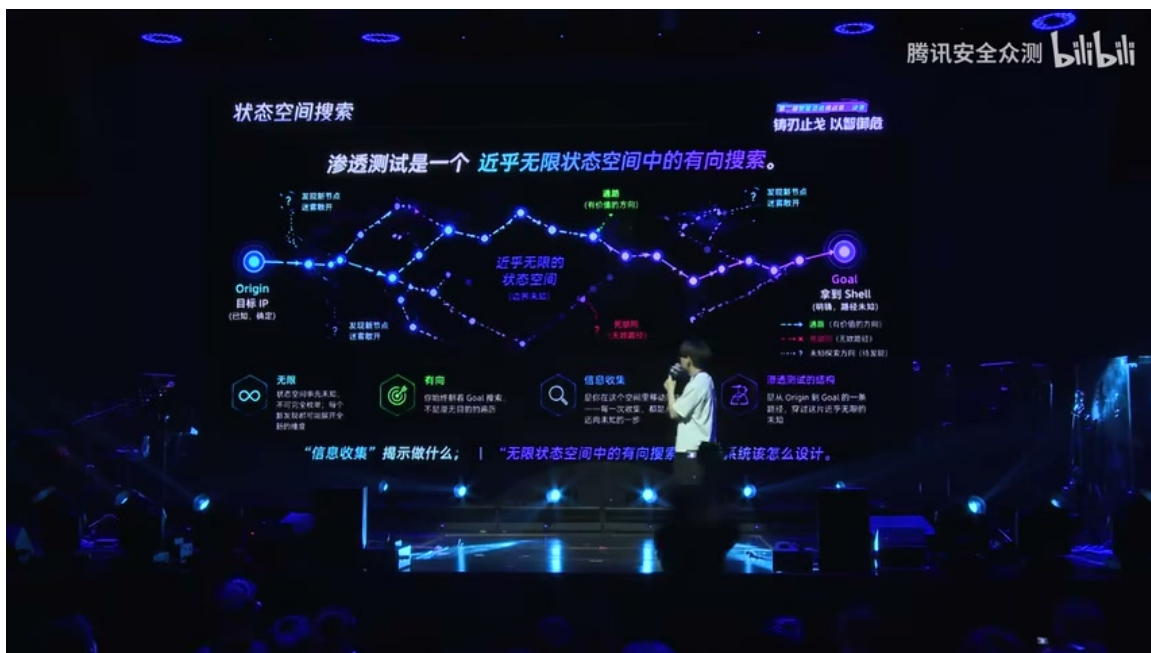


图 1: 状态空间搜索示意图。从 Origin（目标 IP，已知、确定）出发，经过近乎无限的状态空间，沿着有向路径最终抵达 Goal（拿到 Shell/Flag）。图中展示了通路（有价值的方向）、死胡同（无效路径）以及发现新节点后的逐层展开。¹

两个互补的揭示

演讲者在此提出了一个极具洞察力的二分法：

- ”信息收集”揭示我们在渗透测试中要做什么——每一步的具体动作。
- ”无限状态空间中的有向搜索”揭示我们的系统应该怎么设计——整体架构应该支持什么、约束什么、优化什么。

¹ 视频画面时间区间：01:03-01:20

前者是战术层面的指导，后者是战略层面的框架。Cairn 系统的设计，正是从后者出发。

1.3 通用问题求解引擎的设计起点

直觉先行：不同领域，同一结构

渗透测试并非唯一符合上述结构的领域。让我们看看以下问题：

- **漏洞挖掘**：起点是目标程序/协议，终点是确认可利用的漏洞，中间路径是无数次模糊测试、逆向分析、构造输入的尝试。
- **数学证明**：起点是已知公理和定理，终点是待证命题，中间路径是无数次推理、构造反例、尝试引用的探索。
- **侦探破案**：起点是案件信息，终点是找出凶手，中间路径是无数次走访、取证、推理的尝试。

这些看似毫不相关的领域，共享着同一个深层结构。

通用问题的共同形式

上述所有问题都可以抽象为以下共同特征：

通用问题求解的三要素

1. **明确的起点** (s_0)：问题的初始条件、已知信息。
2. **明确的达成标准** (s^*)：目标的判定条件，何时算“完成”。
3. **未知的中间路径** (π)：从起点到终点的具体步骤事先未知，需要在探索中动态发现。

用形式化语言表述，即：

$$\forall \mathcal{P} \in \{\text{渗透测试, 漏洞挖掘, 数学证明, ...}\}, \quad \mathcal{P} = (\mathcal{S}, s_0, s^*, \delta) \quad (5)$$

这意味着，如果我们能设计一个系统，有效地求解这种结构的问题，它就不再局限于渗透测试，而成为一个通用问题求解引擎。

Cairn（衍迹）的设计动机

这正是 Cairn（衍迹）系统的设计起点。”Cairn” 在英文中意为路标——荒野中由旅人堆砌的石堆，指示方向、标记路径。这个名字与系统的核心意境高度契合：

- 在无限的状态空间中，Cairn 不是地图（地图不可能完整绘制），而是路标——标记已经探索过的路径、指示可能的方向。
- 每个被发现的 **Fact**（事实）是一块石头，每个 **Intent**（意图）是一个指向。
- 路标由探索者共同维护，后来者可以沿着前人的标记继续推进。

设计动机总结

如果我们能正确地识别问题的本质结构——**无限状态空间中的有向搜索**——并据此设计系统，那么这套系统就不仅是渗透测试的工具，而是一个**通用问题求解引擎**。Cairn 正是基于这一洞察而构建的。

1.4 本章小结

本章从渗透测试的一个经典回答——”信息收集”——出发，完成了以下认知升级：

1. **范式转换**：将”信息收集”从目的本身重新诠释为”状态空间中的移动方式”，从而把渗透测试从工具链思维提升为搜索问题。
2. **形式化定义**：将渗透测试形式化为四元组 $\mathcal{P} = (\mathcal{S}, s_0, s^*, \delta)$ ，明确了无限状态空间 ($|\mathcal{S}| \rightarrow \infty$) 和有向搜索 (s^* 预先确定) 两个核心特征。
3. **通用化洞察**：识别出渗透测试、漏洞挖掘、数学证明等领域共享的同一深层结构——**明确起点、明确达成标准、未知中间路径**——从而为通用问题求解引擎的设计奠定了理论基础。
4. **设计动机**：Cairn (衍迹) 系统正是基于”无限状态空间中的有向搜索”这一本质抽象而设计的，其命名”路标”也呼应了在未知空间中标记和导航的核心思想。

读者应该记住什么

- 渗透测试的本质不是”信息收集”这个动作本身，而是在**无限状态空间中寻找从起点到终点的有向路径**。
- 信息收集是**状态空间中的移动方式**，不是最终目的。
- 无限（不可穷举）和有向（目标明确）是这个问题的两个不可分割的侧面。
- 这一结构不仅适用于渗透测试，也适用于漏洞挖掘、数学证明等广泛领域，因此可以支撑**通用问题求解引擎**的设计。

2 黑板架构：共享知识空间的设计范式

2.1 侦探围坐黑板的隐喻与黑板架构的历史

让我们先想象一个画面：一群侦探围坐在同一块黑板前破案。黑板上写满了案件相关的信息——已确认的时间线、嫌疑人的不在场证明、现场发现的物证……侦探们有的在黑板前沉思，有的出门调查取证，回来后再把新发现写回黑板。所有人共享同一块黑板，所有人的推理都基于黑板上当前呈现的信息。

这个画面中有几个关键结构值得注意：

- **节点**：黑板上每一个已确认的信息块，代表一个已经发现的事实；
- **边**：连接节点的箭头，代表一个侦探的意图——”接下来要去验证什么”；
- **问号**：如果某件事情还没做，边上就标着一个问号；做了之后，问号变成确定的事实；

- **便利贴**：黑板边缘贴着几张便利贴，上面写着主侦探的洞见或外部提示，不参与图内部的推理，但作为外部参考。



图 2：侦探围坐黑板破案的隐喻——节点代表事实，边代表意图

2

这个隐喻的核心在于：**所有参与者共享同一个知识空间**。没有人拥有私有信息，所有的发现和意图都写在黑板上，任何人随时可以看到全局态势。这正是 Cairn 系统设计的起点。

黑

板隐喻的核心洞察：节点代表已发现的事实，边代表待执行的意图。问号表示”尚未验证”，确认后变为事实。整个系统通过共享黑板实现多 Agent 的间接协调。

有趣的是，演讲者先凭直觉设计出了这套模式，后来查阅资料才发现它早已有名字——**黑板架构** (Blackboard Architecture)。

黑板架构的历史渊源

1970 年代，卡内基梅隆大学 (CMU) 的 Hearsay-II 语音识别系统首次提出了黑板架构。该系统让多个独立的知识源 (Knowledge Source) 围绕一块共享黑板工作：每个知识源读取黑板上的当前状态，贡献自己的专业知识，没有中央调度器统一指挥。这种”各自读取当前状态，各自贡献新知识”的范式，与侦探围坐黑板的画面如出一辙。

同一个底层道理，在完全不同的领域被一再发现——从语音识别到侦探破案，再到渗透测试——这让演讲者更加确信这个抽象方向是对的。

²视频画面时间区间：01:46-02:13

³视频画面时间区间：02:13-02:38

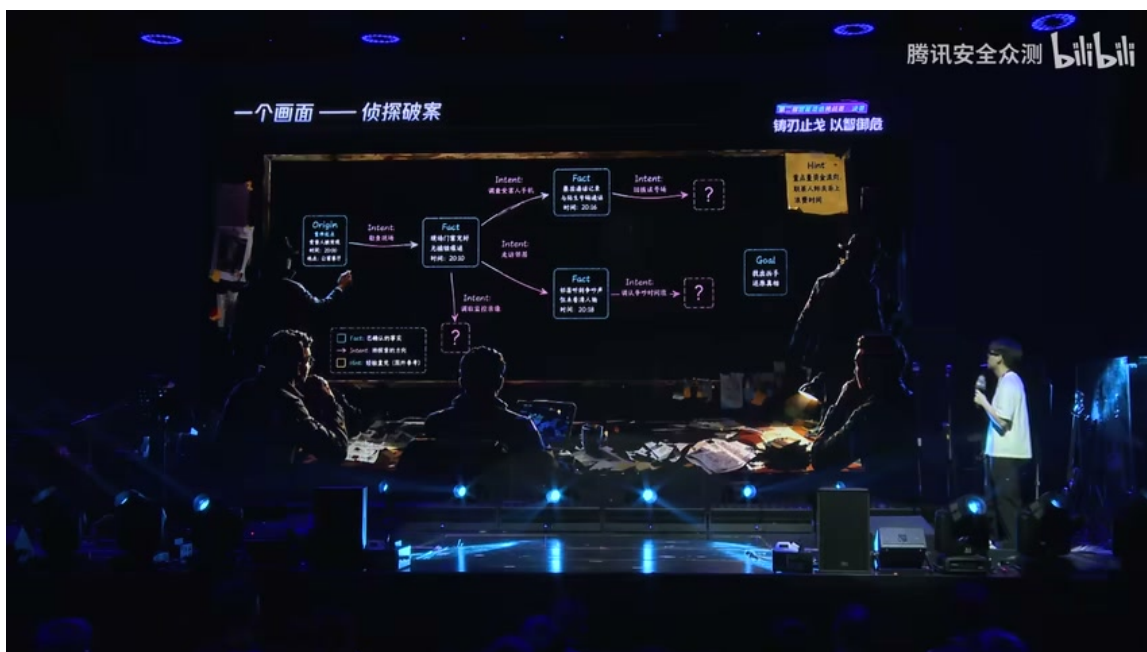


图 3: 黑板架构的历史——1970 年代 CMU 的 Hearsay-II 语音识别系统

3

2.2 Cairn 黑板的三类核心元素：Factor、Intent、Hint

在 Cairn 系统的黑板中，所有元素被严格划分为三类。让我们继续用侦探破案的隐喻来理解它们：

2.2.1 Factor（关键事实）：已确认的板书

Factor（关键事实）：黑板上用粉笔写下的已确认信息，代表一个已经发现、已经验证的事实。在渗透测试语境中，例如”目标 IP 的 22/80/443 端口开放”、”Web 服务使用 nginx 1.18”。

Factor 是黑板上的”已确认板书”。它是确定的、不可撤销的——一旦写上去，就成为后续所有推理的基础。侦探们不会质疑黑板上已经确认的事实，只会基于它们继续推导。

2.2.2 Intent（探索意图）：粉笔画出的问号

Intent（探索意图）：黑板上用粉笔画出的问号和箭头，代表一个要做的事情、一个待验证的假设。例如”探测 HTTP 服务版本”、”尝试 SQL 注入”。

Intent 是”待办事项”。它还没有被执行，只是被推理出来、认为值得尝试。当某个 Agent Worker 执行了这个 Intent，它就会产生新的 Factor（执行结果），然后这个 Intent 就完成了它的使命。

2.2.3 Hint（外部提示）：便利贴上的线索

Hint（外部提示）：黑板边缘的便利贴，代表来自图外的高维输入。例如人类操作员的洞见、题目本身给出的提示、或者外部工具的初步分析结果。

Hint 的特殊之处在于它**不参与图内部的推理循环**。它不会作为节点或边出现在知识图中，而是作为外部参考被 Agent 读取。人类通过 Hint 注入意图，但具体的推理和探索仍由 Agent 自主完成——这正是”任务式指挥”（Mission Command）的精髓：指挥官只下达意图，下级根据现场态势自主决策。

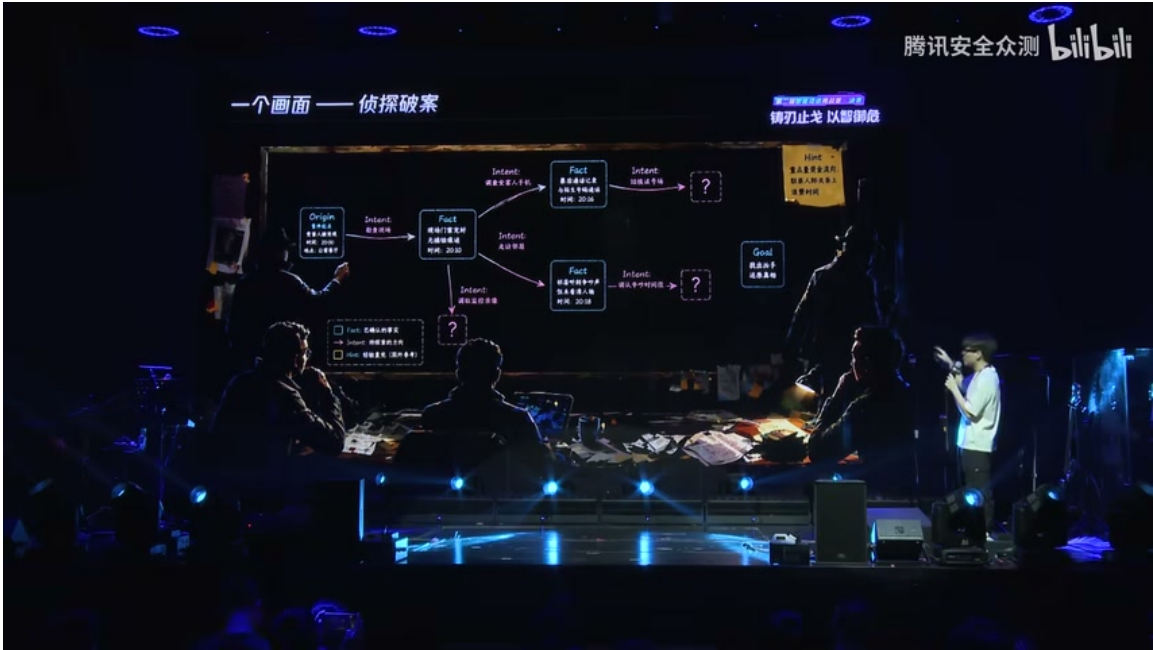


图 4: Cairn 黑板的三类核心元素：Factor、Intent、Hint

4

常

见误解：Hint 不是 Factor。很多人会把”题目提示”或”人类指令”直接当作事实写入知识图，这是错误的。Hint 是外部参考，它提供方向但不提供确定性。Agent 需要自行验证 Hint 所指向的内容，将验证结果作为 Factor 写入黑板。

2.3 从黑板到图：知识结构的图化表示

有了 Factor、Intent 和 Hint 这三类元素，黑板上的内容就不再是一堆散乱的笔记，而是一张结构清晰的有向图。

2.3.1 知识图的形式化定义

我们将黑板上的知识结构形式化为一张有向图：

⁴视频画面时间区间：02:38-03:22

$$\mathcal{G} = (V, E)$$

其中：

- V 是顶点集合，每个顶点 $v \in V$ 代表一个 **Factor**（关键事实）；
- E 是边集合，每条有向边 $e = (u, v) \in E$ 代表一个 **Intent**（探索意图），表示“基于事实 u ，通过执行意图 e ，期望发现事实 v ”。

进一步地，我们定义三类元素的集合：

$$\mathcal{F} = \{f_1, f_2, \dots, f_n\} \quad (\text{Factor 集合, 对应顶点 } V)$$

$$\mathcal{I} = \{i_1, i_2, \dots, i_m\} \quad (\text{Intent 集合, 对应边 } E)$$

$$\mathcal{H} = \{h_1, h_2, \dots, h_k\} \quad (\text{Hint 集合, 外部提示})$$

知

识图 $\mathcal{G} = (V, E)$ 中，顶点 V 对应 Factor 集合 $\mathcal{F}$ ，边 E 对应 Intent 集合 $\mathcal{I}$ 。Hint 集合 $\mathcal{H}$ 作为外部输入，不直接参与图的结构，但影响 Agent 的推理方向。

2.3.2 从起点到终点的有向路径

这张知识图有一个明确的起点和一个明确的终点：

- **起点** (Origin)：题目给出的所有初始信息，例如目标 IP、端口范围、题目描述等。起点是一个特殊的 Factor，标记为 f_{origin} 。
- **终点** (Goal)：获取 Flag。终点也是一个特殊的 Factor，标记为 f_{goal} 。

$$\text{起点 } f_{\text{origin}} \xrightarrow{i_1} f_1 \xrightarrow{i_2} f_2 \xrightarrow{i_3} \dots \xrightarrow{i_k} \text{终点 } f_{\text{goal}}$$

整个渗透测试的过程，就是从起点出发，沿着知识图不断生长，最终到达终点的过程。每一次 Agent 的执行，都是在图上添加新的 Factor（节点）和新的 Intent（边）。

2.3.3 Cairn（路标）命名的意境

Cairn 的渊源

Cairn 在英文中原意是“路标”——登山者用石头垒起的小堆，用来标记路径，指引后来者。在 Cairn 系统中，每一个 Factor 就是一块石头，每一条 Intent 就是向前探索的一步。Agent 沿着前人留下的路标继续探索，同时为后来者垒起新的路标。

这个命名与系统的意境高度契合：

- 每个新 **Factor** 是一块已知的石头；
- 每条 **Intent** 是踩向未知的一步；
- 整张图从 **origin** 出发，向 **goal** 方向生长；



图 5: 从黑板到图——知识结构的有向图表示，从 origin 向 goal 生长

5

- Agent 沿着前人的 Cairn 继续探索，同时为后来者垒起新的 Cairn。

常

见误解：知识图不是预先设计好的。很多人误以为需要事先定义好所有节点和边的结构，然后让 Agent 去“填充”。实际上，知识图是**动态生长**的——起点和终点是预先定义的，但中间的节点和边完全由 Agent 在运行过程中推理和探索产生。图的结构是涌现的，不是预设的。

2.4 本章小结

本章从侦探围坐黑板的直觉隐喻出发，逐步过渡到黑板架构的技术抽象，最终形式化为知识图 $\mathcal{G} = (V, E)$ 的数学表示。核心要点如下：

1. **黑板架构**是一种让多个独立知识源围绕共享状态空间工作的设计范式，最早可追溯至 1970 年代 CMU 的 Hearsay-II 语音识别系统。
2. Cairn 黑板包含三类核心元素：
 - **Factor** ($\mathcal{F}$): 已确认的关键事实，对应图中的顶点；
 - **Intent** ($\mathcal{I}$): 待执行的探索意图，对应图中的有向边；
 - **Hint** ($\mathcal{H}$): 外部提示，作为参考输入但不参与图结构。
3. 黑板上的元素天然构成一张有向图 $\mathcal{G} = (V, E)$ ，从起点（题目信息）向终点（获取 Flag）生长。图的中间结构是动态涌现的，而非预先设计。
4. **Cairn**（路标）的命名精准地捕捉了系统的本质：每个 Factor 是一块石头，每条 Intent 是向前的一步，整张图是登山者留给后来者的路径标记。

⁵ 视频画面时间区间：03:22-04:08

在下一章中，我们将深入探讨 Agent 的工作循环——Agent 如何读取黑板上的图、如何推理出新的 Intent、以及如何执行探索并将结果写回黑板。

3 Agent 工作循环：OODA 与三任务类型

本

章核心命题：Cairn 系统中没有任何一行代码告诉 Agent “渗透测试应该怎么做”。Agent 看到的只有当前图的结构，以及三种任务指令之一。所有策略都从图的态势中动态涌现。

在前一章建立的黑板架构之上，本章将深入剖析 Agent 的工作循环机制。我们将回答一个关键问题：当多个无身份的 Worker 共享同一块黑板时，它们如何协调工作、推进渗透测试？答案是一套极简的任务类型系统与 OODA 决策循环的同构映射。

3.1 三类任务类型：Bootstrap、Reason、Explore

Cairn 系统中的每个 Agent Worker 只执行三种任务之一。用符号表示，任务类型集合为：

$$\mathcal{T} = \{\tau_B, \tau_R, \tau_E\}$$

其中 τ_B 表示 Bootstrap 任务， τ_R 表示 Reason 任务， τ_E 表示 Explore 任务。下面分别阐述。

3.1.1 Bootstrap：直接尝试解决

定义 3.1 (Bootstrap 任务 τ_B)。拿到渗透测试场景或通用问题后，Agent 首先尝试**直接解决**。若成功，则直达终点；若失败，则记录所有尝试过程与发现的新事实，写回图中。

Bootstrap 的核心思想是“先试试再说”。在很多简单题目中，直接尝试可能就能拿到 Flag；即便失败，尝试过程中产生的信息（如端口扫描结果、页面响应特征、错误提示等）也会成为图上的新节点，为后续推理提供素材。

Bootstrap 的命名来源

“Bootstrap”一词源自计算机科学中的“自举”概念——程序不借助外部工具，依靠自身启动。在 Cairn 中， τ_B 意味着 Agent 在没有先验路径的情况下，仅凭题目描述和入口信息就开始行动。

3.1.2 Reason：读取全图并推理下一步

定义 3.2 (Reason 任务 τ_R)。Agent 读取当前图的完整结构——所有已确认的 *Fact* 节点、所有待执行的 *Intent* 边、所有外部 *Hint*——然后**推理下一步应该做什么**，输出一个新的意图 (*Intent*)。

Reason 是系统的“大脑”。它不执行任何实际操作，而是站在全局视角审视当前态势：

- 哪些 Fact 节点已经被确认？
- 哪些 Intent 边尚未执行？
- 哪些 Fact 可以组合产生新的推论？

- 距离终点（获取 Flag）还有多远？

基于这些分析，Reason 任务输出一个或多个新的 Intent，指明下一步的探索方向。

3.1.3 Explore：执行推理出的路径

定义 3.3 (Explore 任务 τ_E). *Agent* 执行 *Reason* 推理出的 *Intent*，进行**实际探索**，将执行结果（成功或失败、新发现的事实）写回图中。

Explore 是系统的”手脚”。它将 Reason 产生的抽象意图转化为具体操作：

- 若 Intent 是”尝试 SQL 注入”，Explore 任务会构造 Payload 并发送请求；
- 若 Intent 是”读取配置文件”，Explore 任务会执行文件读取操作；
- 若 Intent 是”利用已知漏洞提权”，Explore 任务会运行 Exploit。

执行完成后，无论成败，Explore 都必须将结果写回图——成功则新增 Fact 节点，失败也可能产生有价值的负面信息。

常见误解：三类任务对应三类 Agent

初学者容易误认为 Cairn 有三类专门的 Agent——一个负责 Bootstrap、一个负责 Reason、一个负责 Explore。**这是错误的**。Cairn 中的 Worker 没有身份区分，任何 Worker 都可能被分配任何任务。任务类型是在**运行时**由 Dispatcher 根据图的态势动态决定的，而非在设计时固化到 Agent 角色中。

3.2 OODA 循环：观察、分析、决定、执行

三种任务类型并非孤立执行，它们构成了一个完整的决策循环——OODA 循环。

[

OODA 循环] OODA 循环 (Observe-Orient-Decide-Act) 源自美国空军上校 John Boyd 的军事理论，描述了一个快速决策与行动反馈的闭环：

1. **Observe (观察)**：感知环境信息
2. **Orient (分析)**：理解态势、建立心智模型
3. **Decide (决定)**：基于分析做出决策
4. **Act (执行)**：实施决策并观察结果

3.2.1 三类任务与 OODA 的映射

Cairn 的三任务系统与 OODA 循环存在精确的同构关系：

这一映射揭示了 Cairn 设计的深层结构：每个 Agent Worker 的微观循环，正是 OODA 循环的一个实例。多个 Worker 在黑板上的交替执行，形成了宏观层面的持续迭代。

表 1: 三类任务与 OODA 循环的映射

OODA 阶段	Cairn 任务	图上的操作
Observe (观察)	τ_B Bootstrap	直接感知场景, 记录初始发现
Orient (分析)	τ_R Reason	读取全图结构, 推理下一步
Decide (决定)	τ_R Reason	输出新的 Intent (决策)
Act (执行)	τ_E Explore	执行 Intent, 写回结果

军事理论中的任务式指挥

OODA 循环在军事领域有一个重要应用——**任务式指挥** (Mission Command)。其核心原则是：上级指挥官只下达**意图** (Intent)，下级部队根据现场态势**自主决策**具体行动方案。Cairn 的设计与此同构：人类操作员通过 Hint 注入高层意图，Agent Worker 根据图的态势自主进行 Reason 和 Explore，无需人类下发每一步的详细指令。

3.2.2 循环的数学表达

设第 t 轮迭代时图的状态为 $G_t = (V_t, E_t)$ ，其中 V_t 为 Fact 节点集合， E_t 为 Intent 边集合。则 OODA 循环可形式化为：

$$G_{t+1} = \tau_E(\tau_R(\tau_B(G_t)))$$

更一般地，Bootstrap 只在初始阶段执行一次（或在新场景触发时执行），后续的循环主要由 Reason 和 Explore 交替驱动：

$$G_{t+1} = \tau_E(\tau_R(G_t)), \quad t \geq 1$$

每次循环后，图的状态 G_{t+1} 至少包含 G_t 的所有信息，并可能新增 Fact 节点和 Intent 边——系统从不遗忘，只会积累。

3.3 真实题目解题过程演示

为了直观理解三任务类型与 OODA 循环的协作方式，下面展示一个线上真实题目的完整解题过程。

3.3.1 起点与终点的定义

- **起点**：题目的所有信息，包括目标 IP 地址、端口、题目描述、附件等。这些信息在图上形成初始 Fact 节点。
- **终点**：获取全部 Flag。这是系统预定义的终止条件。

3.3.2 阶段一：Bootstrap 尝试

系统首先向 Agent 下发 τ_B 任务：

”直接尝试解决这道题目，拿到 Flag。”

Agent 开始行动：扫描端口、访问 Web 页面、尝试常见漏洞利用。正如预期，**Bootstrap 失败了**——这道题目并非简单到可以直接秒解。

但 Bootstrap 并非毫无收获。在尝试过程中，Agent 发现了若干新事实：

- 目标开放 80 端口，运行一个 Web 应用；
- 应用存在一个登录页面；
- 页面源码中泄露了一个可疑的 API 端点。

这些发现作为新的 Fact 节点被写回图中，同时产生若干待验证的 Intent 边（如“尝试 SQL 注入登录”、“访问泄露的 API 端点”）。

3.3.3 阶段二：Reason-Explore 循环

Bootstrap 完成后，系统进入 $\tau_R \rightarrow \tau_E$ 的交替循环。

第一轮 Reason：Agent 读取全图，看到：

- 已知 Fact：Web 应用、登录页面、API 端点泄露；
- 待执行 Intent：SQL 注入尝试、API 端点访问等；
- 终点目标：获取 Flag。

Agent 推理后输出新的 Intent：“先访问泄露的 API 端点，获取更多信息。”

第一轮 Explore：Agent 执行上述 Intent，访问 API 端点后发现：

- API 返回了一个用户列表，包含用户名和加密后的密码哈希；
- 哈希算法可识别为 MD5。

新 Fact 写回图中，图的状态更新为 G_2 。

第二轮 Reason：Agent 再次读取全图，看到新增了用户列表和密码哈希。推理后输出 Intent：“尝试破解 MD5 哈希，获取登录凭据。”

第二轮 Explore：Agent 执行哈希破解，成功获取一组有效的用户名和密码。

第三轮 Reason：Agent 读取全图，看到已有登录凭据和登录页面。推理后输出 Intent：“使用获取的凭据登录系统。”

第三轮 Explore：Agent 执行登录操作，成功进入系统后台，在后台页面中发现 Flag。

关

键观察：整个过程中，没有任何一行代码告诉 Agent“先扫端口、再 Web 渗透、再提权”。Agent 的每一步决策都来自于对当前图结构的 Reason 分析。同样的 Worker，在图的不同态势下，会做出完全不同的决策。

3.3.4 复杂题目的真实图结构

上述题目较为简单，Reason-Explore 循环仅进行了三轮。对于复杂题目，真实的图结构会更加庞大。

如图 6 所示，复杂题目的图可能具有以下特征：

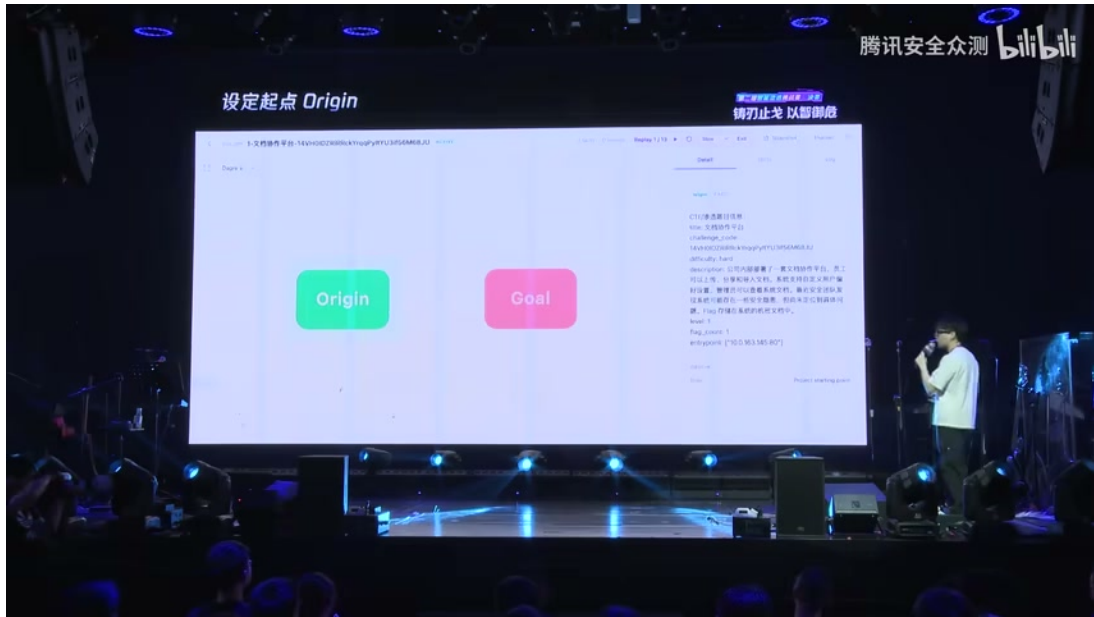


图 6: 复杂题目的真实图结构示意。图中包含大量 Fact 节点（已确认事实）、Intent 边（待执行/已执行的意图），以及从多个上游节点组合产生的新推论。

- **多分支并行**：从一个 Fact 节点可以衍生出多条独立的探索路径，由多个 Worker 并行执行；
- **跨分支组合**：来自不同分支的 Fact 可以组合产生新的 Intent（如“用 A 分支获取的密码登录 B 分支发现的接口”）；
- **深层嵌套**：某些路径需要多轮 Reason-Explore 才能推进，形成深度递进结构；
- **死胡同与回溯**：部分 Intent 执行后证明不可行，对应的边标记为失败，但不从图中删除——负面信息同样有价值。

常见误解：图是预先设计好的

有人可能误以为复杂题目的图是系统设计时预先定义好的“攻略路线图”。**事实恰恰相反**：图是在解题过程中**动态生长**的。每一道新题目、每一次新发现，都会在图上留下独特的结构。Cairn 系统的设计目标不是覆盖所有可能的图，而是提供一套机制，让图能够在运行时自然涌现。

3.4 本章小结

本章阐述了 Cairn 系统中 Agent 的工作循环机制，核心要点如下：

1. **三任务类型**：Agent 只执行三种任务——Bootstrap（直接尝试）、Reason（读取全图并推理）、Explore（执行推理出的路径）。任务类型集合为 $\mathcal{T} = \{\tau_B, \tau_R, \tau_E\}$ 。
2. **OODA 同构**：三任务系统与军事理论中的 OODA 循环（Observe-Orient-Decide-Act）精确对应。Bootstrap 对应观察，Reason 对应分析与决策，Explore 对应执行。
3. **运行时涌现**：Agent Worker 没有身份区分，任务类型由 Dispatcher 根据图的态势动态分配。分工不是设计时固化的，而是运行时从图的结构中涌现的。

4. **图动态生长**: 解题过程中, 图从初始的少量 Fact 节点不断生长, 新增节点和边由 Bootstrap 的发现和 Reason-Explore 循环驱动。复杂题目的图可能包含大量并行分支和跨分支组合。
5. **核心设计原则**: 没有任何一行代码告诉 Agent 渗透测试应该怎么做。Agent 的决策唯一来源是当前图的结构和任务指令——这是 Cairn 与传统预定义流程 Agent 架构的根本区别。

在下一章中, 我们将进一步探讨 Cairn 的工程架构——Curator Server、Dispatcher 和 Agent Container 如何协作, 以及这套架构如何支撑本章所述的工作循环。

4 系统工程架构: Current Server、Dispatcher 与容器化 Agent

前三章我们建立了 Cairn 的核心理论框架——以图为核心的状态空间搜索范式, 以及 Agent 的三类任务循环 (Bootstrap、Reason、Explore)。本章将深入系统的工程实现层面, 剖析支撑这一范式运行的三层架构: Current Server、Dispatcher 与容器化 Agent。理解这三层如何分工协作, 是把握 Cairn 系统设计的钥匙。

4.1 三层工程架构: Current Server、Dispatcher、容器

Cairn 的系统工程架构自上而下分为三个层次, 每一层职责清晰、边界明确, 共同构成了一个松耦合、高内聚的分布式推理系统。

三层架构概览

- **Current Server**: 协议层/黑板层, 维护图的核心数据结构, 保证一致性
- **Dispatcher**: 调度中间层, 负责读图、写图、调用具体容器
- **容器 (Container)**: 执行层, 搭载各类 Agent (Cloud Code、Cursor、轻量 Agent)

4.1.1 Current Server: 协议层与黑板层

Current Server 是整个系统的核心协议层, 它直接对应第二章所述的”黑板架构”的工程实现。Speaker 将其明确定义为:

”Current Server 其实是一个协议, 或者说是我这套系统的黑板。它维护三个基本数据结构, 它只管图的一致性, 它不做任何推理。”

Current Server 的职责可以概括为以下几点:

第一, 维护三个基本数据结构。这三个结构分别对应黑板上的三种元素: 已确认的 **Fact** (事实节点)、待探索的 **Intent** (意图边)、以及外部注入的 **Hint** (提示信息)。Current Server 确保这三类数据在并发读写场景下的一致性。

第二, 只管一致性, 不做推理。这是 Current Server 最核心的设计原则。它类似于数据库的事务管理器——负责 ACID 特性, 但不理解业务逻辑。所有的推理、决策、策略选择都被推至上层的 Agent。这种”无知”恰恰是其强大之处: 它不会因推理逻辑的改变而需要修改, 从而获得了极高的稳定性。

第三, 作为协议层暴露接口。Current Server 定义了图操作的标准协议, 包括节点的增删改查、边的状态迁移、以及 Hint 的注入接口。Dispatcher 和容器都通过这一协议与图交互, 而非直接操作底层存储。

用形式化的语言描述，Current Server 维护的图态势可以表示为：

$$\theta = f(\mathcal{G})$$

其中 $\mathcal{G}$ 表示当前图的结构（节点集合与边集合）， θ 表示由图结构所蕴含的全局态势。Current Server 负责维护 $\mathcal{G}$ 的一致性，而 θ 的解读则交由上层的 Reason 任务完成。

4.1.2 Dispatcher：读图与写图的中间层

Dispatcher 位于 Current Server 与容器之间，是连接“数据”与“计算”的桥梁。它的核心职责包括：

读取图状态。Dispatcher 从 Current Server 获取当前图的完整快照（或增量更新），并将其序列化为特定容器可理解的格式。不同容器可能需要不同的上下文组织方式——例如 Cloud Code 可能需要更详细的代码级上下文，而轻量 Agent 可能只需要高层意图描述。

调度任务分发。当 Reason 任务产生新的 Intent 节点时，Dispatcher 负责将这些意图分配给合适的 Worker 执行。它维护一个 Worker 池，根据当前系统负载、Worker 能力、以及任务优先级进行调度决策。

写入执行结果。Worker 完成任务后，Dispatcher 将新的 Fact 或更新的 Intent 状态写回 Current Server。这一写操作必须遵循 Current Server 定义的协议，以保证图的一致性。

Dispatcher 本身也不做推理——它的“智能”体现在调度策略上，而非对图内容的理解。这种设计保持了架构的清晰分层：Current Server 管数据，Dispatcher 管调度，容器管执行。

4.1.3 容器：搭载各类 Agent 的执行环境

容器层是系统中最贴近具体执行环境的层次。Speaker 提到：

“容器里面搭载了各种的 Agent，比如 Cloud Code、Cursor，以及自己实现的一些轻量 Agent。”

容器的设计体现了 Cairn 对“异构执行”的支持。不同类型的 Agent 有不同的能力边界和适用场景：

- **Cloud Code**：具备强大的代码理解与生成能力，适合需要深度代码分析的任务
- **Cursor**：擅长交互式编程与快速迭代，适合探索性编码任务
- **轻量 Agent**：系统自研的精简 Agent，上下文开销小，适合简单、高频的推理任务

容器为 Agent 提供隔离的执行环境，包括文件系统沙箱、网络访问控制、以及资源限制。这种隔离既保证了安全性（一个容器的崩溃不会影响其他容器），也支持了并发执行（多个容器可以同时运行不同的 Explore 任务）。

容器化设计的工程意义

容器化 Agent 的设计使得 Cairn 可以灵活地集成不同厂商、不同版本的 AI 模型服务。当新的模型（如 GPT-5、Claude 4）发布时，只需开发对应的容器适配层，而无需修改 Current Server 或 Dispatcher 的核心逻辑。这种“可插拔”的架构是 Cairn 能够持续演进的技术基础。

三层架构的关系可以用一个简洁的公式概括：

$$\text{Cairn 系统} = \underbrace{\text{Current Server}}_{\text{数据一致性}} + \underbrace{\text{Dispatcher}}_{\text{任务调度}} + \underbrace{\sum_i \text{Container}_i}_{\text{异构执行}}$$

4.2 Hint 注入机制与人类指挥官角色

在三层架构之上，Cairn 设计了一套精巧的外部输入机制——Hint 注入，以及与之配套的人机协作范式——任务式指挥（Mission Command）。

4.2.1 Hint：图外的高维输入

Hint 是 Cairn 中一个独特而重要的概念。Speaker 指出：

”人类的意图是通过这个 Hint 注入进去的，包括题目的提示也是通过 Hint 注入进去的。”

Hint 的本质是**图外的高维输入**。它不参与图内部的推理循环，但作为一种外部参考影响着 Agent 的决策。在第二章的侦探比喻中，Hint 就相当于主侦探的洞见——它不会被写在黑板上成为案件事实的一部分，但会指导侦探们的调查方向。

Hint 的来源有两类：

人类意图。操作 Cairn 的安全分析师可以通过 Hint 向系统注入高层策略。例如：“重点关注 SQL 注入漏洞”、“先尝试弱口令”、“注意检查备份文件”。这些 Hint 不会替代 Agent 的自主推理，但会为其提供先验倾向。

题目提示。在 CTF 竞赛等场景中，题目本身可能附带提示信息。这些提示通过 Hint 机制注入系统，使 Agent 能够利用题目设计者提供的线索，而不会将其误认为已发现的事实。

Hint 的设计体现了 Cairn 对“人类在环”（Human-in-the-Loop）的深刻理解：人类不应该被排除在自动化系统之外，但也不应该被降格为简单的“启动按钮”。Hint 机制让人类以**轻量、非侵入**的方式参与系统运行——既提供了价值，又不破坏 Agent 的自主性。

4.2.2 任务式指挥：Mission Command

Speaker 将 Cairn 的人机协作模式与军事理论中的“任务式指挥”（Mission Command，德语：Auftragstaktik）进行了类比：

”这种其实与军事理论中有一个叫做任务式指挥。意思就是说人类作为指挥官，只下达意图，然后下级的 Worker 会根据现场的图里的态势自主进行决策。人类不会下发详细的指令。”

任务式指挥的核心原则是**意图导向而非指令导向**。传统指挥模式中，上级需要制定详细的行动计划（“第一步做什么、第二步做什么”），下级只需机械执行。而任务式指挥中，上级只说明**要达到什么目标和为什么要达到这个目标，如何达到**则完全交由下级根据现场情况自主决定。

在 Cairn 中，这一原则体现为：

- **人类指挥官：**通过 Hint 注入意图（“找到所有 flag”、“优先尝试 Web 路径”）
- **Worker：**读取当前图态势 θ ，自主决定下一步行动
- **无详细指令：**人类不会告诉 Worker“先用 nmap 扫描端口，再尝试 SQLMap”

任务式指挥 vs. 脚本式指挥

维度	任务式指挥 (Cairn)	脚本式指挥 (传统)
人类输入	高层意图 (Hint)	详细步骤 (脚本)
Agent 角色	自主决策者	机械执行者
适应性	高 (根据态势动态调整)	低 (按预设流程执行)
可扩展性	高 (同一意图适配多场景)	低 (每场景需新脚本)

这种设计使得 Cairn 在面对未知场景时表现出强大的适应性。当系统遇到从未见过的漏洞类型时，脚本式系统会束手无策（因为没有对应的脚本），而任务式系统仍能在“获取 flag”这一意图的指引下，通过自主推理找到新的攻击路径。

4.3 依赖处理与并发执行策略

在多 Worker 协作的场景中，两个核心问题自然浮现：**任务之间的依赖如何处理？多个 Worker 能否并行工作？** Speaker 在评委提问环节对这两个问题给出了清晰的回答。

4.3.1 任务依赖：由 AI 推理动态产生

Cairn 中任务依赖的处理方式与传统 workflow 系统截然不同。Speaker 强调：

“任务的依赖确实是有依赖的。比如你可以看到这个节点，它来自两个上游节点。当然它上游节点，这是由 AI 推理出来的，这不是由系统设定的。”

他以一个具体例子说明：

“AI 觉得这两个——比如这个是拿到了一对密码，这个是找到了一个登录的接口——那两个就可以自然地组合出一个：用这个密码去登录这个接口。”

这一设计的关键洞察在于：**依赖关系不是预先定义的，而是由 AI 根据当前图态势动态推理产生的**。在传统 workflow 系统中，开发者需要显式声明“任务 A 必须在任务 B 之前完成”。而在 Cairn 中，Reason 任务读取当前图状态后，自主判断哪些已发现的事实可以组合产生新的行动意图。

这种动态依赖发现的能力赋予了 Cairn 极强的灵活性。系统无需预先知道“密码”和“登录接口”之间存在关联——当这两个事实同时出现在图中时，Reason 任务会自动识别出它们的组合价值，并生成相应的依赖关系。

形式化地说，若图中已存在事实集合 $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ ，Reason 任务执行一个隐式的组合函数：

$$\text{Deps} = \text{Reason}(\mathcal{F}, \theta) = \{(f_i, f_j) \mid f_i \text{ 与 } f_j \text{ 可组合产生新意图}\}$$

4.3.2 串行推理与并行执行

在并发策略上，Cairn 采用了**串行推理、并行执行**的非对称设计。

Reason 推理是串行的。Speaker 明确说明：

“推理是当整个图有变化的时候，它才推理。所以推理在我这里是串行的，它只在图变化时进行推理。”

串行推理的设计有其深层原因。Reason 任务需要读取**完整的图上下文**，并基于全局态势做出战略决策。如果允许多个 Reason 任务同时运行，它们可能基于相互矛盾的图快照产生冲突的意图，导致系统行为不一致。此外，Reason 任务可能触发 Agentic 探索（如翻阅本地知识库），这进一步要求串行化以避免资源竞争。

Explore 执行是并行的。与串行推理形成鲜明对比，Explore 任务可以高度并行化：

”执行是可以并行的。图比较复杂的时候，它会同时从一个节点衍生出两条不同的边，那这两条不同的边，会有两个 Worker 去同时并行执行。”

并行执行的前提是**无依赖冲突**。当 Reason 任务产生多个独立的 Intent（即它们之间没有依赖关系，且不操作共享资源）时，Dispatcher 可以将这些 Intent 分发给不同的 Worker ω_i 并行执行：

$$\{\omega_1, \omega_2, \dots, \omega_k\} \parallel \text{Explore}(\{e_1, e_2, \dots, e_k\})$$

其中 e_j 表示第 j 条待执行的边（Intent）。

常见误解：Cairn 是单线程系统

有观点认为 Cairn 的串行推理设计意味着整个系统是单线程的，这会限制其在复杂渗透场景中的效率。这是一种误解。

澄清：Cairn 的串行性仅限于 Reason 阶段，而 Explore 阶段可以充分利用多 Worker 并行。在复杂场景中，一次 Reason 调用可以产生**多条**待探索的边，这些边随后被并行分发执行。因此，系统的整体吞吐量取决于 Explore 的并行度，而非 Reason 的串行性。

此外，Speaker 指出 Reason 任务本身可以进行”Agentic 探索”——在推理过程中进行简单的知识库翻阅，然后**一次性产生多个意图**。这意味着串行推理并不等同于”一次只尝试一条路径”。

4.3.3 串并混合的执行模型

综合以上分析，Cairn 的执行模型可以概括为以下循环：

1. **触发：**图发生变化（新的 Fact 写入或 Intent 状态更新）
2. **串行 Reason：**单个 Reason 任务读取完整图上下文，产生一个或多个新 Intent
3. **依赖解析：**系统检查新 Intent 之间的依赖关系
4. **并行 Explore：**无依赖冲突的 Intent 被分发给多个 Worker ω_i 同时执行
5. **结果回写：**各 Worker 将执行结果写回图，触发下一轮循环

这一模型可以用图7示意。

这种设计的工程价值在于：它在**一致性**（串行 Reason 保证决策质量）和**效率**（并行 Explore 提升吞吐量）之间取得了优雅的平衡。正如 Speaker 在回答评委时所确认的——推理过程确实”类似于单线程”，但执行过程可以”并行分发”。

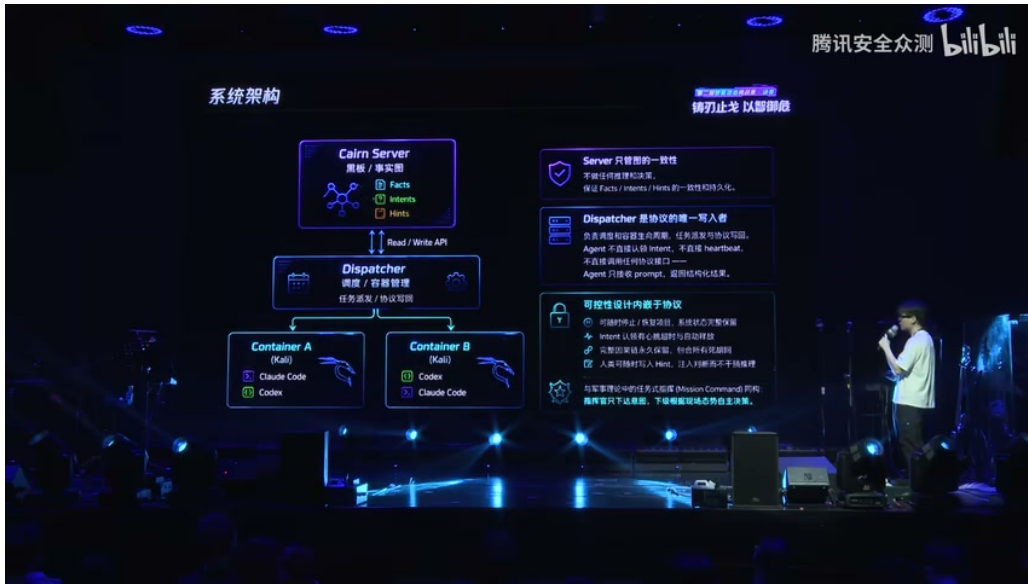


图 7: Cairn 串并混合执行模型: Reason 串行、Explore 并行

4.4 本章小结

本章深入剖析了 Cairn 系统的三层工程架构及其运行机制:

Current Server 作为协议层/黑板层, 维护 Fact、Intent、Hint 三个基本数据结构, 只管图的一致性而不做任何推理。它是整个系统的”单一事实来源”, 为上层提供了可靠的数据基础。

Dispatcher 作为调度中间层, 负责读图、写图、调用容器。它在 Current Server 的数据协议与容器的执行需求之间进行适配和转换, 是系统灵活性的关键所在。

容器层搭载各类异构 Agent (Cloud Code、Cursor、轻量 Agent), 提供隔离的执行环境, 支持多 Worker 并发运行。

Hint 注入机制以非侵入的方式将人类意图和题目提示引入系统, 配合**任务式指挥** (Mission Command) 范式, 实现了”人类下意图、Worker 自主决策”的协作模式。

串行推理、并行执行的并发策略在保证决策一致性的前提下最大化了系统吞吐量。任务依赖由 AI 根据图态势动态推理产生, 而非预先硬编码, 这使得 Cairn 能够适应从未见过的问题场景。

这三层架构共同支撑了 Cairn 的核心理念: **Worker 没有身份, 只有任务; 任务不从角色定义中来, 而从图的态势中来**。在下一章中, 我们将对比 Cairn 与传统多 Agent 架构的本质区别, 进一步阐明这一设计理念的深层意义。

5 设计哲学: Less is More 与涌现式智能

本章是全篇的核心与灵魂。如果说前几章介绍了 Cairn 系统的”是什么”和”怎么做”, 那么本章将深入探讨”为什么”——为什么 Cairn 选择了一条与传统多 Agent 架构截然不同的道路, 为什么”少”反而比”多”更有力量, 以及这种设计哲学背后关于智能本质的深刻思考。

5.1 传统多 Agent 架构 vs Cairn 架构的核心差异

核心命题

Cairn 与传统多 Agent 架构的区别, 不在于有没有分工, 而在于**分工从哪里来**。

5.1.1 传统架构：设计时预定义的角色分工

在传统多 Agent 系统中，设计者会在系统构建之初就为不同的 Agent 分配固定的角色与职责。如图8所示，典型的传统架构包含：

- **信息搜集 Agent**：负责端口扫描、目录枚举、指纹识别等侦察任务
- **Web 漏洞利用 Agent**：负责 SQL 注入、XSS、文件上传等 Web 攻击
- **后渗透 Agent**：负责权限维持、横向移动、内网渗透
- **横向移动 Agent**：专门负责在内网环境中扩展控制范围



图 8：传统多 Agent 架构与 Cairn 架构的核心对比：左侧为预定义角色的传统方案，右侧为动态任务生成的 Cairn 方案。图中关键洞察：“Worker 没有身份，只有任务。任务从图里来，不从角色定义里来。”⁶

这种架构的运作方式是：一个总调度器（Orchestrator）根据预设规则，将不同类型的任务分发给对应角色的 Agent。每个 Agent 只执行自己”擅长”的工作，彼此通过消息传递或共享内存进行协作。

传统架构的隐性代价

角色分工看似合理，实则必然带来三个问题：

1. **信息孤岛**：每个 Agent 只掌握自己领域的上下文，跨领域关联需要额外的信息传递层
2. **上下文缺失**：信息在传递过程中被过滤、翻译、损耗，原始语义层层衰减
3. **僵化**：面对未在设计时预见的场景，预定义角色束手无策

⁶ 视频画面时间区间：08:18-08:30

5.1.2 Cairn 架构：运行时动态产生的任务

Cairn 采取了完全不同的路径。在 Cairn 中，所有的 Worker 是完全同质的——它们没有身份，没有预设角色，只有当前被分配的任务。任务不是来自设计时的角色定义，而是来自运行时图的当前态势：

- 当图上的 Fact 发生变化，触发新一轮 Reason，产生新的 Intent
- 新的 Intent 被写入图后，Dispatcher 将其分发给空闲的 Worker 执行 Explore
- 同一个 Worker 这一轮可能做 Reason，下一轮可能做 Explore
- 任务的边界由图的状态动态划定，而非由人类设计者静态划定

关键洞察

Worker 没有身份，只有任务。任务从图里来，不从角色定义里来。

这种差异的本质可以用一个公式表达。设传统架构中的分工为 R_{design} (设计时角色集合), Cairn 中的分工为 $T_{runtime}$ (运行时任务集合)：

$$R_{design} = \{r_1, r_2, \dots, r_n\}, \quad r_i \text{ 由人类设计者在构建时定义}$$

$$T_{runtime} = f(G_t), \quad G_t \text{ 为时刻 } t \text{ 的图状态}$$

前者是静态集合，后者是图状态的动态函数。当 G_t 变化时， $T_{runtime}$ 随之变化，无需任何人工干预。

5.1.3 分工来源的本质差异

表 2: 传统架构与 Cairn 架构的分工来源对比

维度	传统多 Agent 架构	Cairn 架构
分工来源	设计时预定义	运行时动态产生
Worker 身份	固定角色（信息搜集/Web 利用/后渗透）	无身份，只有当前任务
上下文共享	跨 Agent 消息传递，存在过滤损耗	直接读写共享黑板，零损耗
适应性	受限于预定义角色覆盖范围	由图态势自然涌现，无边界
信息孤岛	必然存在	不存在

演讲者在此处提出了一个振聋发聩的论断：

设计智能 vs 设计人类的局限

传统架构以为自己在设计智能，其实它只是在复刻人类的局限。

人类需要分工，因为人类的认知带宽有限、专业知识有限、记忆容量有限。但 AI 没有这些局限——一个 LLM 可以同时理解 Web 漏洞、密码学和系统安全。给 AI 强行分配”信息搜集 Agent”和”Web 利用 Agent”的角色，不是发挥了 AI 的优势，而是把人类的组织方式强加给了 AI。

5.2 蚁群隐喻：信息素间接协调与涌现行为

既然 Cairn 的 Worker 之间不直接通信，那么它们如何协调？演讲者在此引入了一个精妙的生物学隐喻——蚁群觅食行为。

5.2.1 蚂蚁的间接协调机制

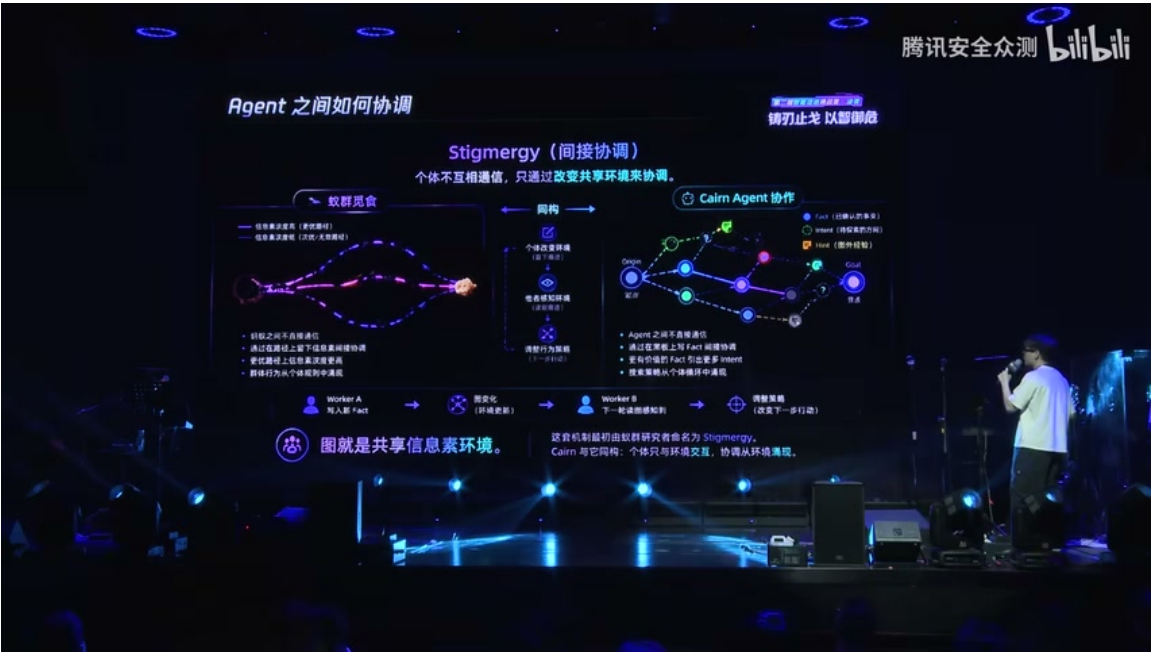


图 9: 蚁群隐喻与 Cairn 的 Stigmergy 机制对比：左侧为蚁群通过信息素间接协调找到最优路径，右侧为 Cairn Agent 通过黑板（共享信息素环境）间接协调推进渗透测试。⁷

蚂蚁在寻找食物时展现了一种令人惊奇的集体智能：

1. 蚂蚁之间从不点对点直接交流
2. 每只蚂蚁在行进路径上留下信息素（Pheromone）痕迹
3. 更短的路径上蚂蚁往返更快，信息素浓度更高
4. 后续蚂蚁倾向于选择信息素浓度更高的路径
5. 群体行为从个体简单规则中**涌现**——没有任何一只蚂蚁”知道”最优路径，但整个蚁群找到了它

Stigmergy（间接协调）

Stigmergy 是蚁群研究者对这一现象的命名，指**个体不互相通信，只通过改变共享环境来协调**的机制。在自然界中，信息素就是共享环境；在 Cairn 中，黑板（事实图）就是共享环境。

5.2.2 Cairn 与蚁群的同构性

Cairn 的设计与蚁群觅食行为具有深刻的结构同构性：

⁷ 视频画面时间区间：09:40-09:55

表 3: 蚁群觅食与 Cairn 渗透测试的同构映射

维度	蚁群觅食	Cairn 渗透测试
共享环境	信息素痕迹	黑板（事实图 G ）
个体行为	感知信息素浓度，选择路径	读取图态势，决定下一步行动
环境修改	留下信息素	写入新 Fact 或 Intent
协调方式	间接协调（Stigmergy）	间接协调（Stigmergy）
群体智能	最优路径涌现	渗透策略涌现

5.2.3 信息素浓度的数学表达

我们可以将黑板上的信息积累形式化为信息素浓度函数。设图的边集为 E ，每条边 $e \in E$ 上的信息素浓度为：

$$\varphi : E \rightarrow \mathbb{R}^+$$

其中 $\varphi(e)$ 表示边 e 上积累的信息素浓度。在 Cairn 中，这对应于：

- 已确认 Fact 的边上信息素浓度高（确定性高）
- 待探索 Intent 的边上信息素浓度待验证
- 无效路径的信息素浓度自然衰减（被标记为失败）

黑板作为共享信息素环境，记为 Φ ：

$$\Phi = (V, E, \varphi, \mathcal{F}, \mathcal{I}, \mathcal{H})$$

其中 $\mathcal{F}$ 为 Fact 集合， $\mathcal{I}$ 为 Intent 集合， $\mathcal{H}$ 为 Hint 集合。所有 Worker 通过读写 Φ 实现间接协调，无需任何点对点通信。

Cairn 的协调原则

Cairn Worker 之间绝不直接通信。它们只通过在黑板上写事实（Fact）和意图（Intent）进行间接协调。整体的渗透测试推进策略和搜索策略，都从这种简单的个体循环中涌现出来。

5.3 Less is More：去除而非堆叠的设计哲学

5.3.1 一个令人震惊的事实

Cairn 系统在整个实现中：

- 0 个渗透工具 MCP
- 0 个安全领域 RAG
- 0 个渗透流程定义
- 0 种预定义 Agent 角色

⁸视频画面时间区间：10:56-11:10



图 10: Less is More 设计哲学: Cairn 系统未使用任何渗透工具 MCP、安全领域 RAG、渗透流程定义或预定义 Agent 角色。图中对比了”在问题外面堆解法”的复杂架构与”在问题内部找抽象”的简单架构。⁸

这在当前 AI 安全 Agent 的潮流中几乎是不可想象的。大多数团队在面对渗透测试这一复杂问题时，第一反应是”堆工具”——加一个 MCP 层来调用 Nmap、Burp Suite、Metasploit；加一个 RAG 层来检索 CVE 知识库；加一个流程定义层来规范攻击步骤；加多个 Sub-Agent 来覆盖不同攻击面。

复杂架构的陷阱

复杂架构是在问题外面堆解法：

- 有问题？加一层
- 缺能力？写 Sub-Agent
- 新场景？加新角色

这种方式直觉自然、门槛低，但**每多一层，系统就脆弱一分**。每一层都是新的故障点、新的维护负担、新的认知复杂度。

5.3.2 寻找本质与最小表达

Cairn 的设计者选择了相反的方向——不是向外堆叠，而是向内挖掘：

Less is More 的核心追问

1. 渗透测试的本质是什么？
2. 状态空间的最小表达是什么？
3. 什么可以被去掉而不损失能力？

演讲者给出的答案是：

- 渗透测试的本质 = 无限状态空间中的有向搜索
- 状态空间的最小表达 = Fact + Intent + Hint 构成的图
- 必须看穿本质，才能知道什么可以去掉

5.3.3 去掉比加上更难

深刻的洞察

去掉一个东西，比加一个东西的理解深度要更大。

加法只需要知道“这个工具能做什么”；减法需要知道“这个问题的本质是什么，什么才是真正不可或缺的”。加法堆叠的是解决方案，减法提炼的是问题的核心结构。

Cairn 的设计者不是不知道 MCP、RAG、预定义流程的存在——而是经过深度思考后，判断它们不是这个问题的本质组成部分。渗透测试不需要一个专门的“信息搜集 Agent”，因为信息搜集本身就是搜索过程的一部分；不需要一个专门的 RAG 层，因为 LLM 本身已经具备安全知识；不需要预定义流程，因为每一步的行动应该由当前态势动态决定。

Less is More 的深层含义

“Less is More”不只是设计风格，也是看到本质后的自然结果。当你真正理解了渗透测试的本质是“无限状态空间中的有向搜索”，当你发现 Fact+Intent+Hint 已经足以表达这个状态空间，那么所有额外的层都是冗余的——不是被刻意去除的，而是自然不需要的。

5.4 赛场表现、评委问答与技术反思

5.4.1 赛场表现：AK 全场的背后

Cairn 系统在 TCH 第二届比赛中的表现极具戏剧性：

- **线上排名第四**：因赛前开局配置失误，前期抢分阶段落后
- **全场唯一 AK**：最终成为唯一解出全部 54 道题目 Flag 的队伍
- **赛前零测试**：比赛当天凌晨 4 点才第一次调通整个系统
- **五天消耗**：约 10.9 亿 Token，约 7692 元人民币

⁹视频画面时间区间：12:20-12:35

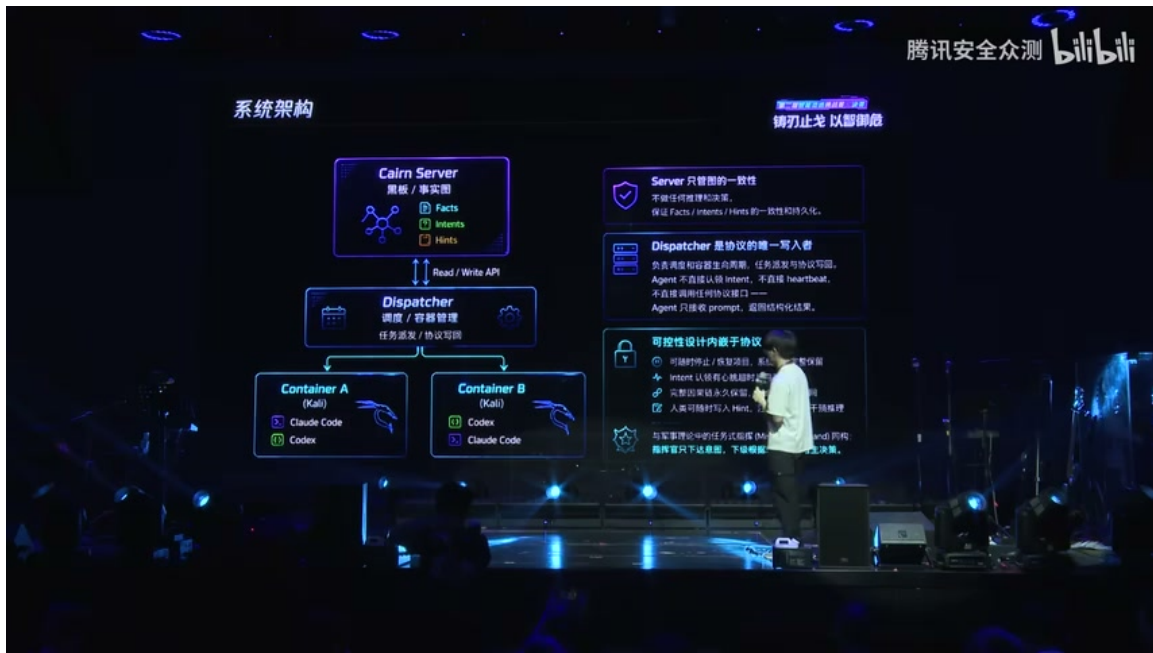


图 11: 比赛表现数据: TCH 第二届, 610 支战队、1345 名选手参赛。Bytex 战队线上排名第四, 但为全场唯一 AK (All-Killed, 全部解出) 所有题目的队伍。五天比赛消耗约 10.9 亿 Token、7692 元人民币。⁹

这组数据揭示了一个深刻的悖论: 一个赛前几乎未经测试、比赛当天凌晨才第一次跑通的系统, 却成为了全场唯一 AK 的队伍。这恰恰证明了 Cairn 架构的鲁棒性——它依赖的不是精细调优的预设流程, 而是对问题本质的正确抽象。只要核心机制 (图搜索 + OODA 循环 + Stigmergy 协调) 正确运行, 具体的实现细节即使粗糙, 也能在赛场上展现出强大的适应性。

5.4.2 评委问答一: 外层严格协议 vs 提示词约束

评委提问: 为什么不基于 Claude Code 或 Cursor 直接构建意图, 而要独立出来一层?

演讲者回答:

如果基于 Claude Code 或 Cursor, 需要通过它们的提示词池 (Prompt Pool) 或 Hook 机制来实现。但这些机制的约束相当弱, 无法满足整个系统的结构需求。因此我希望外层有一个严格限定的协议、严格的框架去调度, 而不是让 Cloud Code 通过 AI 的提示词遵守去做整个过程。

这一回答揭示了 Cairn 设计的一个关键原则: **可控性设计内嵌于协议**。系统的正确性不能依赖于 LLM 对提示词的“自觉遵守”, 而必须依赖于外层的严格协议约束。协议规定了:

- Agent 只接收 prompt, 返回结构化结果
- Agent 不直接认领 Intent, 不直接 heartbeat
- Dispatcher 是协议的唯一写入者
- Server 只管图的一致性, 不做任何推理和决策

5.4.3 评委问答二：模型选择

评委提问：最后用的模型是什么？

演讲者回答：GPT 和 Cloud Code/Cursor。

Cairn 的上层系统构建意图后，将小的意图交给 Cloud Code 或 Codex 执行。这体现了 Cairn 架构的**分层智能**设计：上层负责战略层面的图推理和意图生成（需要长上下文、全局视野），下层负责战术层面的代码执行和工具调用（需要精确的代码生成能力）。

5.4.4 评委问答三：依赖关系与并行执行

评委提问：怎么处理任务之间的依赖关系？执行是严格顺序的还是可以并发？

演讲者回答：

任务的依赖是由 AI 推理出来的，不是由系统设定的。比如一个节点来自两个上游节点——一个是拿到了一对密码，一个是找到了一个登录接口——AI 自然地推理出“用这个密码去登录这个接口”的意图。

关于并发：推理是串行的，只在图变化时进行推理。但执行是可以并行的。当图比较复杂时，会同时从一个节点衍生出两条不同的边，会有两个 Worker 去同时并行执行。

串行推理，并行执行

Cairn 的执行模型可以概括为：

- **Reason (推理)：**串行，仅在图状态变化时触发，获取完整上下文进行 agentic 推理
- **Explore (探索)：**可并行，多个 Worker 可同时执行不同的 Intent 边
- **一次推理可产生多条边：**Reason 过程可以输出多个待探索的 Intent，由 Dispatcher 并行分发

评委进一步追问：这是否意味着推理过程限制了同一时间只能尝试一个思路？

演讲者的回答是：推理可以一次产生多个边（多个待探索的 Intent），然后再通过并行分发去执行。Reason 本身不是静态的——它可以进行简单的 agentic 探索（翻阅本地知识库、poke 一下环境），然后给出多条路径建议。这是一个动态过程，而非静态的单线程决策。

5.5 本章小结

本章从三个递进层次阐述了 Cairn 的设计哲学：

第一，分工来源的革命。Cairn 与传统多 Agent 架构的根本差异不在于是否分工，而在于分工从何而来。传统架构的分工来自设计时的人类预定义，Cairn 的分工来自运行时图态势的动态产生。Worker 没有身份，只有任务；任务从图里来，不从角色定义里来。

第二，涌现式协调机制。Cairn 借鉴了蚁群的 Stigmergy 机制——个体不直接通信，只通过改变共享环境（黑板/信息素环境 Φ ）来间接协调。信息素浓度函数 $\varphi: E \rightarrow \mathbb{R}^+$ 形式化了这一机制。群体智能从简单个体规则的重复中涌现，无需任何中心化的指挥。

第三，Less is More 的设计哲学。Cairn 未使用任何 MCP、RAG 或预定义流程，不是因为不知道它们的存在，而是因为经过深度思考后判断它们不是本质。去掉一个东西比加一个东西需要更大的理解深度——必须先看穿本质，才知道什么可以去掉。

第四，实战验证。赛前零测试、凌晨才调通的系统，成为全场唯一 AK 的队伍。五天消耗十亿 Token 和七千多元人民币。评委问答进一步澄清了外层严格协议的必要性、模型分层使用的策略，以及串行推理与并行执行的混合执行模型。

贯穿本章的核心论点是：**传统架构以为自己在设计智能，其实它只是在复刻人类的局限。**真正的智能设计，应该释放 AI 的潜力，而非用人类的组织方式束缚它。

6 总结与延伸

6.1 演讲者的实质性总结

在演讲的最后，演讲者用一张总结幻灯片（图12）概括了 Cairn 系统的三个核心要点：



图 12: 演讲者总结幻灯片：三个核心要点——（1）渗透测试是无限状态空间中的有向搜索；（2）完整上下文的 Worker，动态生成的任务；（3）在问题内部找抽象，而非在问题外面堆解法。¹⁰

1. **渗透测试是无限状态空间中的有向搜索。**状态空间搜索只需要已确认的发掘（Fact）、待探索的方向（Intent）、因果链保留。黑板架构是这个需求的最自然载体。这不是渗透测试独有的结构——Cairn 是通用问题求解引擎，渗透测试只是第一个靶场。
2. **完整上下文的 Worker，动态生成的任务。**没有预设角色，任务由图的当前态势在运行时产生。Agent 之间通过黑板间接协调（Stigmergy），无需互相通信。完整图作为每次任务的上下文，让跨阶段推理自然发生。
3. **在问题内部找抽象，而非在问题外面堆解法。**0 个 MCP、0 个 RAG、0 种预定义 Agent 角色。可控性设计内嵌于协议。Less is More，不只是设计风格，也是看到本质后的自然结果。

¹⁰ 视频画面时间区间：11:00-11:15

6.2 结构化提炼

6.2.1 核心论点

核心论点

设计智能，而非设计人类的局限。

传统多 Agent 架构将人类的组织方式（角色分工、层级汇报、专业领域隔离）投射到 AI 系统上，这恰恰束缚了 AI 的优势。Cairn 通过去除预设角色、去除直接通信、去除外部工具依赖，让 AI 在共享的图态势中自然涌现协作行为——这才是对 AI 能力的真正释放。

6.2.2 核心机制

Cairn 系统的核心机制可以归纳为以下公式化表达：

状态表示：

$$G = (V, E, \mathcal{F}, \mathcal{I}, \mathcal{H})$$

其中 V 为节点（状态）， E 为边（转移）， $\mathcal{F}$ 为已确认事实， $\mathcal{I}$ 为待探索意图， $\mathcal{H}$ 为人类/题目提示。

OODA 循环：

$$\text{Observe} \rightarrow \text{Orient} \rightarrow \text{Decide} \rightarrow \text{Act}$$

对应 Cairn 中的 Bootstrap $\rightarrow$ Reason $\rightarrow$ Explore 循环。

Stigmergy 协调：

$$\varphi : E \rightarrow \mathbb{R}^+, \quad \Phi = \text{共享黑板（信息素环境）}$$

Worker 通过读写 Φ 实现间接协调，无需点对点通信。

任务动态生成：

$$T_{runtime} = f(G_t), \quad \text{非} \quad T_{design} = \{r_1, r_2, \dots, r_n\}$$

6.2.3 实践启示

从 Cairn 的设计中，我们可以提炼出以下对 AI 系统设计的实践启示：

1. **先问本质，再问工具。**在引入任何工具、框架、中间件之前，先追问：这个问题的本质是什么？最小表达是什么？什么可以去掉？
2. **共享上下文优于消息传递。**与其让 Agent 之间互相发送消息（带来信息损耗和同步复杂度），不如让所有 Agent 读写同一份共享状态（黑板）。
3. **动态分工优于静态角色。**让任务从问题态势中自然产生，而非从设计者的预设中分配。这样系统才能适应未预见的场景。
4. **协议约束优于提示词约束。**系统的正确性应该由严格的协议保证，而非依赖 LLM 对提示词的“自觉遵守”。
5. **涌现优于编排。**复杂的群体行为可以从简单的个体规则中涌现。与其精心编排每一步，不如设计好基础机制和共享环境，让智能自然生长。

6.3 概念压缩与跨章节关联

6.3.1 核心概念的层次结构

Cairn 系统的概念可以按抽象层次组织如下：

表 4: Cairn 概念层次结构

层次	概念	作用
哲学层	Less is More、涌现式智能	设计指导思想
隐喻层	蚁群 Stigmergy、侦探黑板	直觉理解框架
抽象层	无限状态空间有向搜索	问题本质建模
结构层	Fact/Intent/Hint、图 G	状态空间最小表达
机制层	OODA 循环、动态任务生成	系统运行规则
协议层	严格协议、可控性内嵌	正确性保障
实现层	GPT/Cloud Code、Dispatcher	具体技术选型

6.3.2 跨章节关联

- 第 1 章（问题本质）提出的”无限状态空间中的有向搜索”，是第 5 章 Less is More 哲学的理论基础——正因为理解了这一本质，才能判断什么是必要的、什么是冗余的。
- 第 2 章（黑板架构）介绍的 Fact/Intent/Hint 三元组，是第 5 章 Stigmergy 机制的载体——黑板就是共享信息素环境 Φ 。
- 第 3 章（OODA 循环）描述的 Bootstrap/Reason/Explore，是第 5 章动态任务生成的具体实现——任务从图态势中动态产生，正是 OODA 循环的运转结果。
- 第 4 章（系统架构）介绍的 Cairn Server/Dispatcher/Container 分层，是第 5 章”外层严格协议”的技术落地——协议约束内嵌于架构设计。

6.4 谨慎的泛化

Cairn 的设计哲学是否可以泛化到渗透测试以外的领域？演讲者本人给出了肯定的回答：

其实不只是渗透测试。像我们漏洞挖掘以及包括数学证明等其他很多类似的问题，它都有共同点：明确的起点、明确的达成标准、未知的中间路径。如果我们做到了正确的系统设计，那我们这套系统就会是一个通用问题的求解引擎。

具备以下特征的问题，都可能适用 Cairn 的设计范式：

- 有明确的起点和终点（Goal）
- 中间路径未知且状态空间巨大
- 每一步探索可以产生新的信息（Fact）
- 信息可以组合产生新的探索方向（Intent）
- 人类可以提供高维度的提示（Hint）

这包括但不限于：数学定理证明、程序自动修复、科学假设生成、复杂系统故障诊断等。

泛化的边界

Cairn 的设计并非万能。以下场景可能不适用：

- 需要严格实时响应的场景（OODA 循环的推理延迟不可接受）
- 状态空间完全离散、无结构可循的纯随机搜索
- 需要精确数学保证的领域（如形式化验证）
- 探索成本极高的场景（每次 Explore 代价过大，无法承受并行试错）

6.5 具体的收获与开放问题

6.5.1 具体收获

学习 Cairn 系统后，读者可以获得以下具体收获：

1. **一个反直觉的设计范式：**在 AI 系统设计中，“少”可以是“多”——去除非本质的层，反而可以获得更强的鲁棒性和适应性。
2. **一个可复用的协调机制：**Stigmergy（间接协调）是一种强大的分布式协调范式，适用于任何需要多个智能体协作但无需直接通信的场景。
3. **一个思考问题的方法：**面对复杂问题，先寻找最小表达（Fact+Intent+Hint），再判断什么可以去掉。
4. **一个实战验证的案例：**赛前零测试、凌晨调通的系统 AK 全场——证明了本质正确的设计比精细调优更重要。

6.5.2 开放问题

Cairn 的设计也留下了若干值得深入思考的开放问题：

1. **推理的串行瓶颈：**Reason 阶段是串行的，这在极端复杂的场景中是否会成为瓶颈？如何在破坏一致性的前提下实现推理的并行化？
2. **Token 消耗的经济性：**五天十亿 Token、七千余元的成本，对于商业应用是否可接受？如何在保持能力的前提下降低 Token 消耗？
3. **图的规模上限：**随着问题复杂度增加，图的规模会指数增长。如何设计图的压缩、摘要和归档机制？
4. **Hint 的自动化：**当前 Hint 依赖人类输入。能否设计自动化的 Hint 生成机制，让系统自己给自己提供高维提示？
5. **从渗透测试到通用求解：**Cairn 声称自己是通用问题求解引擎，但在渗透测试以外的领域（如数学证明）尚未有公开验证。这一泛化是否成立，需要更多实证。
6. **对抗性场景：**在存在对抗性干扰的场景中（如主动防御系统试图误导 Agent），Stigmergy 机制是否仍然鲁棒？

6.5.3 下一步

对于希望借鉴 Cairn 设计哲学的读者，建议的下一步行动：

1. **复现最小系统**：尝试用你最熟悉的语言和框架，实现一个最小化的 Cairn 原型——只包含黑板（图）、三种任务类型（Bootstrap/Reason/Explore）和一个简单的 Dispatcher。
2. **选择一个问题域**：不要从渗透测试开始（需要靶场环境），可以从一个你熟悉的、有明确起点和终点的问题开始（如解谜题、代码审计、CTF 逆向）。
3. **观察涌现行为**：运行系统，观察 Fact 和 Intent 如何在图上自然生长，观察群体行为如何从简单规则中涌现——这比任何理论分析都更有说服力。
4. **做减法实验**：尝试从你的系统中去掉一层（如去掉 RAG、去掉预定义流程），观察能力是否下降。如果不变或反而提升，说明你找到了一个非本质层。

最后的提醒

Cairn 的设计者用一句话总结了他的核心理念：

“Worker 没有身份，只有任务。任务从图里来，不从角色定义里来。”

这句话值得每一位 AI 系统设计者铭记。当你下一次设计多 Agent 系统时，先问自己：我是在设计智能，还是在复刻人类的局限？